

Lista 01 – Exercícios com o REDIS (Práticas de Laboratório).

Profº Ricardo Roberto de Lima

Aluno: Lucca de Sena Barbosa

1. Strings (Manipulação Básica de Chaves)

- **Exercício 01:** Crie uma chave `sistema:nome` com o valor `"DataPlatform"` e recupere o valor salvo.
 - *Comandos:* `SET sistema:nome "DataPlatform" | GET sistema:nome`
- **Exercício 02:** Grave três variáveis de ambiente no mesmo comando: `env:db=postgres`, `env:cache=redis` e `env:queue=rabbitmq`. Em seguida, busque apenas `env:db` e `env:cache`.
 - *Comandos:* `MSET env:db "postgres" env:cache "redis" env:queue "rabbitmq" | MGET env:db env:cache`
- **Exercício 03:** Defina um token de sessão `sessao:usuario:101` com o valor `"xyz123"` que expire automaticamente em **30 segundos**. Em seguida, verifique quanto tempo resta antes da expiração.
 - *Comandos:* `SETEX sessao:usuario:101 30 "xyz123" | TTL sessao:usuario:101`
- **Exercício 04:** Crie um contador de acessos para a página principal `pageviews:home`. Incremente-o em 1 unidade, depois em mais 5 unidades, e por fim decremente 2 unidades.
 - *Comandos:* `INCR pageviews:home | INCRBY pageviews:home 5 | DECRBY pageviews:home 2`

RESPOSTA:

```
127.0.0.1:6379> SET sistema:nome "DataPlatform"
OK
127.0.0.1:6379> GET sistema:nome
"DataPlatform"
127.0.0.1:6379> MSET env:db "postgres" env:cache "redis" env:queue "rabbitmq"
OK
127.0.0.1:6379> MGET env:db env:cache
1) "postgres"
2) "redis"
127.0.0.1:6379> SETEX sessao:usuario:101 30 "xyz123"
OK
127.0.0.1:6379> TTL sessao:usuario:101
(integer) 28
127.0.0.1:6379> TTL sessao:usuario:101
(integer) 23
127.0.0.1:6379> TTL sessao:usuario:101
(integer) 13
127.0.0.1:6379> TTL sessao:usuario:101
(integer) -2
127.0.0.1:6379> INCR pageviews:home
(integer) 6
127.0.0.1:6379> INCRBY pageviews:home 5
(integer) 11
127.0.0.1:6379> DECRBY pageviews:home 2
(integer) 9
127.0.0.1:6379> GET pageviews:home
"9"
127.0.0.1:6379> |
```

2. Hashes (Objetos e Dicionários)

- **Exercício 05:** Crie um perfil de usuário `usuario:201` contendo os campos `nome` ("Carlos"), `email` ("carlos@email.com") e `nivel` ("admin").
 - *Comandos:* `HSET usuario:201 nome "Carlos" email "carlos@email.com" nivel "admin"`
- **Exercício 06:** Recupere apenas o campo `email` do `usuario:201`. Em seguida, recupere todos os campos e valores armazenados para esse usuário.
 - *Comandos:* `HGET usuario:201 email | HGETALL usuario:201`
- **Exercício 07:** No mesmo `usuario:201`, incremente um novo campo `tentativas_login` em 1. Em seguida, liste apenas os nomes de todas as chaves (campos) presentes nesse hash.
 - *Comandos:* `HINCRBY usuario:201 tentativas_login 1 | HKEYS usuario:201`

RESPOSTA:

```
127.0.0.1:6379> HSET usuario:201 nome "Carlos" email "carlos@email.com" nivel "admin"
(integer) 3
127.0.0.1:6379> HGETALL usuario:201
1) "nome"
2) "Carlos"
3) "email"
4) "carlos@email.com"
5) "nivel"
6) "admin"
127.0.0.1:6379> HGET usuario:201 email
"carlos@email.com"
127.0.0.1:6379> HGETALL usuario:201
1) "nome"
2) "Carlos"
3) "email"
4) "carlos@email.com"
5) "nivel"
6) "admin"
127.0.0.1:6379> HINCRBY usuario:201 tentativas_login 1
(integer) 1
127.0.0.1:6379> HKEYS usuario:201
1) "nome"
2) "email"
3) "nivel"
4) "tentativas_login"
127.0.0.1:6379> █
```

3. Lists (Filas e Pilhas)

- **Exercício 08:** Crie uma fila de tarefas `fila:email`. Adicione no final da fila os itens `"email_1"`, `"email_2"` e `"email_3"`.
 - *Comandos:* `R PUSH fila:email "email_1" "email_2" "email_3"`
- **Exercício 09:** Remova e retorne o primeiro elemento da fila `fila:email` (simulando um consumidor FIFO).
 - *Comandos:* `L POP fila:email`
- **Exercício 10:** Liste todos os elementos restantes na fila `fila:email` sem removê-los.
 - *Comandos:* `L RANGE fila:email 0 -1`

RESPOSTA:

```
127.0.0.1:6379> R PUSH fila:email "email_1" "email_2" "email_3"
(integer) 3
127.0.0.1:6379> L POP fila:email
"email_1"
127.0.0.1:6379> L RANGE fila:email 0 -1
1) "email_2"
2) "email_3"
```

4. Sets (Conjuntos Não Ordenados e Sem Duplicatas)

- **Exercício 11:** Crie um conjunto `tags:post:1` e adicione as tags `"dados"`, `"redis"`, `"nosql"` e novamente `"redis"`. Verifique quantos elementos existem no conjunto.
 - *Comandos:* `S ADD tags:post:1 "dados" "redis" "nosql" "redis" | SCARD tags:post:1`
- **Exercício 12:** Crie um segundo conjunto `tags:post:2` com as tags `"redis"`, `"python"` e `"backend"`. Execute o comando para encontrar as tags que aparecem em ambos os posts (interseção).
 - *Comandos:* `S ADD tags:post:2 "redis" "python" "backend" | S INTER tags:post:1 tags:post:2`
- **Exercício 13:** Verifique se a tag `"python"` pertence ao conjunto `tags:post:1`.
 - *Comandos:* `S ISMEMBER tags:post:1 "python"`

RESPOSTA:

```
127.0.0.1:6379> S ADD tags:post:1 "dados" "redis" "nosql" "redis"
(integer) 3
127.0.0.1:6379> SCARD tags:post:1
(integer) 3
127.0.0.1:6379> S ADD tags:post:2 "redis" "python" "backend"
(integer) 3
127.0.0.1:6379> S INTER tags:post:1 tags:post:2
1) "redis"
127.0.0.1:6379> S ISMEMBER tags:post:1 "python"
(integer) 0
```

5. Sorted Sets (Conjuntos Ordenados por Score)

- **Exercício 14:** Crie um leaderboard de jogo `placar:game` inserindo: "alice" (score 1500), "bob" (score 2200) e "carol" (score 1800).
 - *Comandos:* `ZADD placar:game 1500 "alice" 2200 "bob" 1800 "carol"`
- **Exercício 15:** Liste o ranking completo do `placar:game` em ordem decrescente (do maior score para o menor), incluindo os valores dos scores na exibição.
 - *Comandos:* `ZREVRANGE placar:game 0 -1 WITHSCORES`
- **Exercício 16:** Aumente a pontuação de "alice" em 800 pontos e verifique a sua nova posição no ranking.
 - *Comandos:* `ZINCRBY placar:game 800 "alice" | ZREVRANK placar:game "alice"`

RESPOSTA:

```
127.0.0.1:6379> ZADD placar:game 1500 "alice" 2200 "bob" 1800 "carol"
(integer) 3
127.0.0.1:6379> ZREVRANGE placar:game 0 -1 WITHSCORES
1) "bob"
2) "2200"
3) "carol"
4) "1800"
5) "alice"
6) "1500"
127.0.0.1:6379> ZINCRBY placar:game 800 "alice"
"2300"
127.0.0.1:6379> ZREVRANK placar:game "alice"
(integer) 0
127.0.0.1:6379> ZREVRANGE placar:game 0 -1 WITHSCORES
1) "alice"
2) "2300"
3) "bob"
4) "2200"
5) "carol"
6) "1800"
127.0.0.1:6379> |
```

6. Pub/Sub, Transações e Administração

- **Exercício 17 (Pub/Sub):** No terminal 1, inscreva-se no canal `notificacoes`. No terminal 2, publique a mensagem "Novo relatório disponível".

- *Terminal 1:* `SUBSCRIBE notificacoes`

```
127.0.0.1:6379> SUBSCRIBE notificacoes
1) "subscribe"
2) "notificacoes"
3) (integer) 1
Reading messages... (press Ctrl-C to quit or any key to type command)
```

- *Terminal 2:* `PUBLISH notificacoes "Novo relatório disponível"`

```
127.0.0.1:6379> PUBLISH notificacoes "Novo relatório disponível"
(integer) 2
127.0.0.1:6379>
```

Resultado Terminal 1

```
127.0.0.1:6379> SUBSCRIBE notificacoes
1) "subscribe"
2) "notificacoes"
3) (integer) 1
1) "message"
2) "notificacoes"
3) "Novo relatório disponível"
Reading messages... (press Ctrl-C to quit or any key to type command)
```

- **Exercício 18 (Transações):** Inicie um bloco de transação para transferir R\$ 50 da conta:A para a conta:B atomicamente.
 - *Comandos:* `MULTI -> DECRBY conta:A 50 -> INCRBY conta:B 50 -> EXEC`
- **Exercício 19 (Gerenciamento de Chaves):** Verifique se a chave `sistema:nome` existe no banco. Se existir, altere o nome da chave para `sistema:app` e depois remova-a.
 - *Comandos:* `EXISTS sistema:nome | RENAME sistema:nome sistema:app | DEL sistema:app`
- **Exercício 20 (Inspeção do Servidor):** Execute um comando para visualizar informações gerais e estatísticas de memória do servidor Redis. Em seguida, limpe apenas os dados do banco de dados atual.
 - *Comandos:* `INFO memory | FLUSHDB`

RESPOSTA:

```
127.0.0.1:6379> MULTI
OK
127.0.0.1:6379(TX)> DECRBY conta:A 50
QUEUED
127.0.0.1:6379(TX)> INCRBY conta:B 50
QUEUED
127.0.0.1:6379(TX)> EXEC
1) (integer) -50
2) (integer) 50
127.0.0.1:6379> EXISTS sistema:nome
(integer) 1
127.0.0.1:6379> RENAME sistema:nome sistema:app
OK
127.0.0.1:6379> DEL sistema:app
(integer) 1
127.0.0.1:6379> INFO memory
```